Relatório do trabalho da disciplina de Integração de Sistemas de Informação

---

Análise de dados de combates em videojogos

Rafael Rocha Matos - a29343

Licenciatura em Engenharia de Sistemas Informáticos

outubro de 2026



Índice

Índice 2
Lista de abreviaturas 3
Resumo **4**
1. Enquadramento 5
    1.1 Ferramentas utilizadas 5
2. Problema 5
    2.1 Pergunta de negócio 6
    2.2 Dados de entrada 6
    2.3 Requisitos funcionais 6
3. Estratégia utilizada 6
    3.1 Arquitetura lógica 6
    3.2 Estrutura do projeto 7
    3.3 Operadores utilizados 7
4. Transformações 9
    4.1 Transformação tr_jogadores.ktr 9
    4.2 Transformação tr_combates.ktr 9
    4.3 Estratégia do log de rejeições 11
5. Job de orquestração 11
    5.1 Percursos de execução 11
    5.2 Teste do ramo de falha 12
6. Resultados e validação 12
    6.1 Persistência em PostgreSQL 12
    6.2 Ranking obtido 13
    6.3 Limitação do cálculo numérico 13
    6.4 Ficheiros gerados 14
    6.5 Dificuldades e soluções 14
7. Vídeo de demonstração 14
8. Conclusão e trabalhos futuros 15
9. Referências bibliográficas 15
Anexo A — Matriz de cumprimento dos requisitos 16

Lista de abreviaturas

Sigla	Designação
BD	Base de Dados
CSV	Comma-Separated Values
DPS	Dano por Segundo
ETL	Extract, Transform and Load
JSON	JavaScript Object Notation
PDI	Pentaho Data Integration
SQL	Structured Query Language
XML	Extensible Markup Language

Resumo

Este relatório apresenta o desenvolvimento de um processo de Extract, Transform and Load (ETL) destinado a integrar e analisar telemetria de combates de um videojogo.

Os dados de origem encontram-se distribuídos por três ficheiros heterogéneos: CSV, XML e JSON.

O processo foi implementado em Pentaho Data Integration e organizado em duas transformações e num Job de orquestração.

A solução valida endereços de correio eletrónico através de uma expressão regular, trata regiões em falta, normaliza texto, cruza identificadores com dados de catálogo, rejeita combates com duração inválida, calcula o dano por segundo (DPS), agrega o DPS médio por arma e persiste os resultados em PostgreSQL. Dos 20 combates iniciais, 19 foram considerados válidos.

O resultado indica que o MACHADO DE GUERRA apresenta o maior DPS médio gravado, 50,83, mais do dobro da ESPADA LONGA, que ocupa a segunda posição com 19,80. A solução produz ainda ficheiros CSV e um log comum de rejeições, garantindo rastreabilidade e repetibilidade da execução.

1. Enquadramento

O presente trabalho foi desenvolvido no âmbito da Unidade Curricular de Integração de Sistemas de Informação, integrada na Licenciatura em Engenharia de Sistemas Informáticos.

O enunciado propõe a aplicação e experimentação de ferramentas de ETL em cenários de integração ao nível dos dados, exigindo as fases de recolha, transformação e carregamento, bem como a documentação das transformações, do Job e dos resultados obtidos.

O projeto foi realizado individualmente por Rafael Rocha Matos.

A solução recorre ao Pentaho Data Integration para definir as transformações e a sua orquestração, ao PostgreSQL 18 para persistir o resultado agregado e ao Git para versionar os artefactos produzidos.

A arquitetura foi desenhada para evidenciar diversidade de formatos, limpeza e validação, cruzamento de dados, cálculo, agregação, exportação, persistência e tratamento explícito de falhas.

1.1 Ferramentas utilizadas

Ferramenta	Utilização no projeto
Pentaho Data Integration (Kettle)	Criação de tr_jogadores.ktr, tr_combates.ktr e job_principal.kjb.
PostgreSQL 18 e pgAdmin 4	Persistência e consulta da tabela stats_combate na base de dados trabalho_isi.
Git	Controlo de versões e disponibilização do projeto.
Ficheiros locais	Fornecimento das origens CSV, XML e JSON e receção das saídas CSV e TXT.

2. Problema

Um estúdio de videojogos recolhe dados dos combates dos jogadores contra bosses. Contudo, a informação chega distribuída por fontes e formatos diferentes e contém problemas que impedem a sua análise direta.

A telemetria utiliza apenas identificadores para jogadores, armas e bosses; existe um endereço de correio eletrónico inválido, uma região em falta e um combate com duração igual a zero.

Sem tratamento, o cenário poderia originar associações incompreensíveis, perda de rastreabilidade ou uma divisão por zero no cálculo do DPS.

2.2 Dados de entrada

Origem	Conteúdo	Volume e casos de teste
jogadores.csv	ID_Jogador, Nome_Jogador, Email e Regiao.	8 jogadores; 1 e-mail sem “@”; 1 região vazia.
catalogo.xml	Catálogo de armas e bosses.	4 armas e 3 bosses.
telemetria.json	Combates com jogador, arma, boss, dano e duração.	20 combates; o combate 13 tem duração igual a 0.

2.3 Requisitos funcionais

- Importar dados em CSV, XML e JSON.
- Validar e normalizar os dados dos jogadores.
- Substituir os identificadores por designações compreensíveis.
- Excluir do cálculo os combates com duração não positiva e registar o motivo.
- Calcular o DPS e a média de DPS por arma.
- Exportar ficheiros de resultado e persistir o ranking numa base de dados.
- Orquestrar as transformações com percursos distintos de sucesso e falha.

3. Estratégia utilizada

A estratégia segue o modelo ETL em três fases. Na extração são lidas três origens locais e heterogéneas.

Na transformação, os dados são validados, normalizados, cruzados e agregados.

No carregamento, o ranking final é exportado para CSV e gravado em PostgreSQL.

A separação em duas transformações cria uma dependência deliberada: primeiro é produzido jogadores_limpos.csv e só depois este ficheiro é utilizado no cruzamento da telemetria.

3.1 Arquitetura lógica

E — Extração	T — Transformação	L — Carregamento
jogadores.csv catalogo.xml telemetria.json	Regex, nulos, maiúsculas, 3 lookups, filtro de duração, cálculo de DPS e média por arma	jogadores_limpos.csv ranking.csv log_rejeitados.txt stats_combate (PostgreSQL)

Os caminhos foram configurados de forma relativa, através de ${Internal.Entry.Current.Directory}/../data/input/... e ${Internal.Entry.Current.Directory}/../data/output/....

Esta opção evita dependências desnecessárias da localização absoluta do projeto e facilita a sua execução noutra máquina.

3.2 Operadores utilizados

Operador	Qtd.	Finalidade principal
CSV file input	2	Ler jogadores e jogadores limpos.
JSON input	1	Ler $.combates[*].
Get data from XML	2	Ler armas e bosses com XPath distintos.
Regex Evaluation	1	Validar o formato do e-mail.
Filter rows	2	Separar e-mails e validar duração.
Add constants	2	Adicionar o motivo de rejeição.
Select values	2	Uniformizar as colunas do log.
Text file output	4	Gerar saídas CSV/TXT.
If field value is null	1	Preencher regiões vazias.
String operations	2	Converter texto para maiúsculas.
Stream lookup	3	Associar os nomes de jogador, arma e boss.
Calculator	1	Calcular o DPS.
Memory Group By	1	Calcular a média por arma.
Table output	1	Gravar o resultado no PostgreSQL.
Entradas do Job	8	Start, 2 Transformations, 2 Write to log, 2 Abort job e Success.

4. Transformações

4.1 Transformação tr_jogadores.ktr

Esta transformação prepara a dimensão de jogadores antes do cruzamento com a telemetria.

O fluxo começa com a leitura de jogadores.csv. A expressão regular ^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ cria o campo email_valido.

O passo Filter rows encaminha os sete registos válidos para limpeza e o registo inválido para o log de rejeições.

Passo	Tipo	Descrição
CSV file input	CSV file input	Lê data/input/jogadores.csv.
Valida email	Regex Evaluation	Valida o e-mail e cria email_valido.
Filter rows	Filter rows	Encaminha 7 válidos e 1 inválido.
Limparnulos	If field value is null	Substitui Região vazia por “Desconhecido”.
Para Maiusculas	String operations	Converte nome e região para maiúsculas.
jogadores_limpos	Text file output	Grava data/output/jogadores_limpos.csv.
Motivo email invalido	Add constants	Adiciona “email invalido” ao ramo rejeitado.
Campos log email	Select values	Mantém ID_Jogador e Motivo_Rejeicao.
log_rejeitados	Text file output	Acrescenta o registo a log_rejeitados.txt.

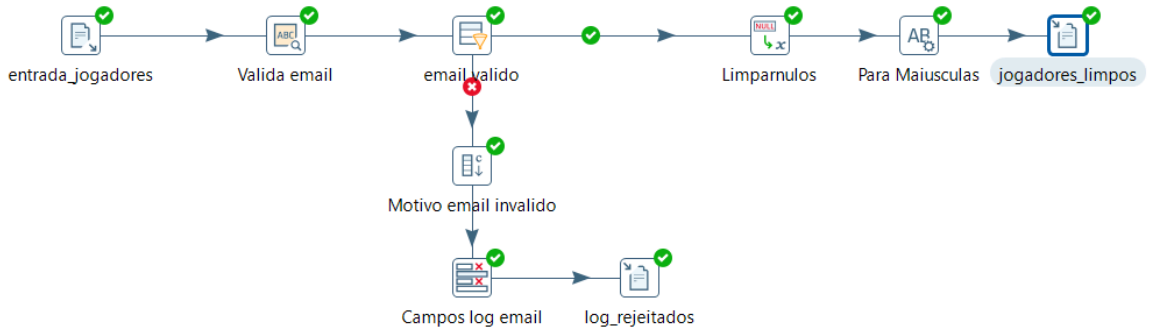


Figura 1 — Captura integral da transformação tr_jogadores.ktr

Resultado: sete dos oito jogadores seguem para jogadores_limpos.csv. O jogador cujo e-mail não contém “@” é registado em log_rejeitados.txt. A região vazia é substituída por “Desconhecido”.

4.2 Transformação tr_combates.ktr

A segunda transformação integra a telemetria com os dados de referência, calcula o DPS e produz o ranking.

Os combates são lidos do caminho JSON $.combates[*]. Em seguida, três Stream lookup associam os identificadores aos nomes do jogador, da arma e do boss.

Caso o jogador não exista em jogadores_limpos.csv, o nome devolvido é “Desconhecido”; desta forma, os combates do jogador rejeitado por e-mail inválido continuam a contribuir para a análise das armas.

Passo	Tipo	Descrição
telemetria	JSON input	Lê os 20 combates de telemetria.json.
jogadores_limpos	CSV file input	Fornece os jogadores válidos ao primeiro lookup.
lookup jogador	Stream lookup	ID_Jogador → Nome_Jogador; omissões tornam-se “Desconhecido”.
XML - Armas	Get data from XML	Lê o catálogo de armas.
lookup arma	Stream lookup	ID_Arma → Nome_Arma.
XML - Bosses	Get data from XML	Lê o catálogo de bosses.
lookup boss	Stream lookup	ID_Boss → Nome_Boss.
Maiusculas	String operations	Normaliza jogador, arma e boss.
Tempo Valido	Filter rows	Aceita apenas duração > 0.
Calculadora DPS	Calculator	DPS = Dano_Causado / Duracao_Combate_Segundos.
Estatisticas Armas	Memory Group By	Agrupa por arma e calcula Media_DPS.
Exportar CSV Ranking	Text file output	Grava ranking.csv.
Gravar stats_combate	Table output	Carrega a tabela e trunca o conteúdo anterior.
rejeicao tempo	Add constants	Adiciona “duração inválida”.
Campos log duracao	Select values	Mantém as duas colunas comuns do log.
log_rejeitados	Text file output	Acrescenta a rejeição ao mesmo ficheiro.

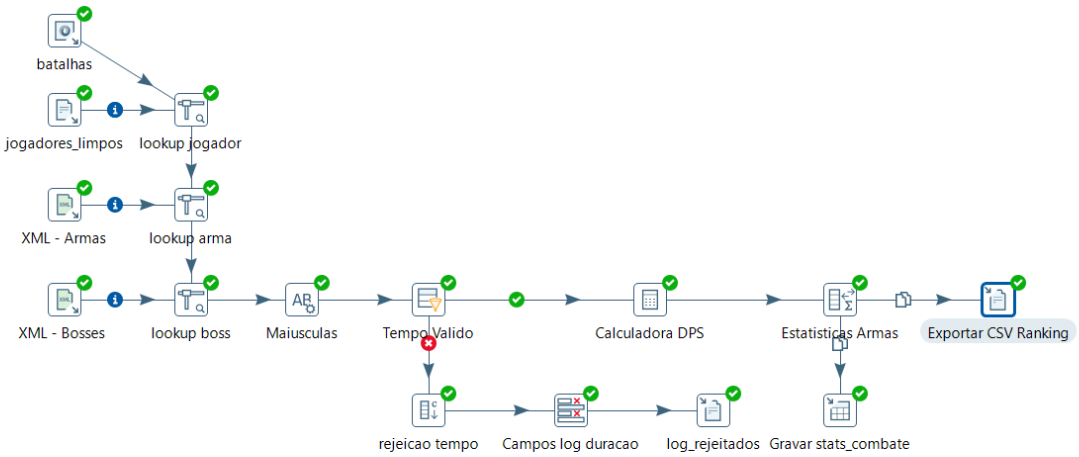


Figura 2 — Captura integral da transformação tr_combates.ktr

O combate 13 é rejeitado por ter duração igual a zero.

A duração não é substituída por 1, pois essa alteração produziria um DPS artificial e falsearia o ranking.

O registo é encaminhado para o log, preservando a rastreabilidade da decisão.

4.3 Estratégia do log de rejeições

As duas transformações escrevem no mesmo ficheiro.

Em ambos os ramos de erro, Add constants cria Motivo_Rejeicao e Select values reduz o esquema a ID_Jogador e Motivo_Rejeicao.

A opção Append encontra-se ativa nos dois Text file output, impedindo que a segunda transformação elimine os registos produzidos pela primeira.

Os dados inválidos não são corrigidos silenciosamente. São separados do fluxo principal e registados com um motivo explícito, permitindo auditoria posterior.

5. Job de orquestração

O ficheiro job_principal.kjb controla a ordem e o estado global da execução. A transformação de jogadores é obrigatoriamente executada em primeiro lugar porque tr_combates.ktr depende do ficheiro jogadores_limpos.csv por ela produzido.

Entrada	Tipo	Função
Start	Start	Inicia o Job.
limpar jogadores	Transformation	Executa tr_jogadores.ktr.
dados DPS	Transformation	Executa tr_combates.ktr.
Success	Success	Termina a execução com sucesso.

Entrada	Tipo	Função
log jogadores	Write to log	Regista a falha na limpeza dos jogadores.
Abort job (1.º)	Abort job	Termina o Job com erro após a primeira falha.
log combates	Write to log	Regista a falha no processamento dos combates.
Abort job (2.º)	Abort job	Termina o Job com erro após a segunda falha.

5.1 Percursos de execução

- Sucesso: Start → limpar jogadores → dados DPS → Success.
- Falha em limpar jogadores: log jogadores → Abort job.
- Falha em dados DPS: log combates → Abort job.

A utilização de dois Abort job permite identificar imediatamente qual das transformações falhou e garante que o estado final do Job é de erro. Isto distingue uma verdadeira orquestração da simples execução manual das transformações.

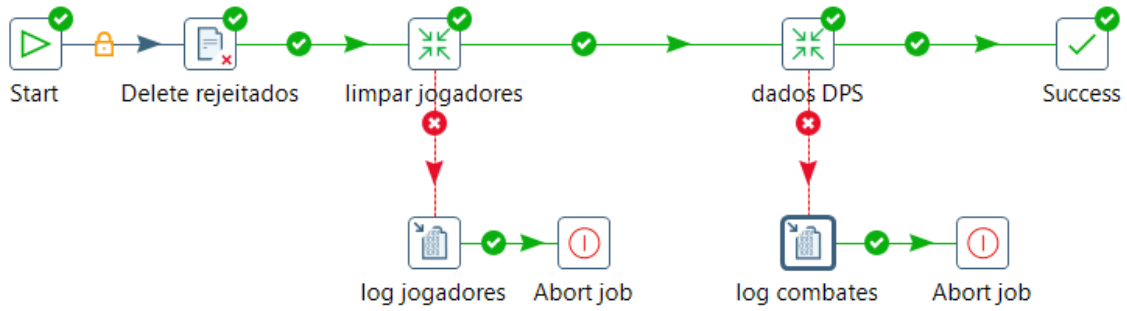


Figura 3 — Captura do Job job_principal.kjb com ramos de sucesso e falha

5.2 Teste do ramo de falha

O ramo de falha deve ser testado alterando temporariamente o caminho de um ficheiro de entrada, por exemplo jogadores.csv. Nessa situação, o Job deve seguir o salto de erro, registar a mensagem em log jogadores e terminar em Abort job. Após o teste, o caminho correto deve ser reposto.

6. Resultados e validação

6.1 Persistência em PostgreSQL

Os resultados são gravados na base trabalho_isi, na tabela stats_combate. A definição utilizada foi:

```
CREATE TABLE stats_combate (
    Nome_Arma VARCHAR(100),
    Media_DPS NUMERIC(10, 2)
);
```

O Table output utiliza Truncate table. Esta configuração torna cada execução idempotente relativamente ao conteúdo do ranking: em vez de acrescentar quatro novas linhas a cada corrida, elimina os resultados anteriores antes do carregamento. A ausência desta opção chegou a produzir 28 linhas após sete execuções.

A consulta de verificação é:

```
SELECT *
FROM public.stats_combate
ORDER BY media_dps DESC;
```

6.2 Ranking obtido

Posição	Arma	DPS médio gravado
1	MACHADO DE GUERRA	50,83
2	ESPADA LONGA	19,80
3	CAJADO ARCANO	14,25
4	ARCO SOMBRIO	12,25

O MACHADO DE GUERRA é a arma com maior DPS médio. O valor gravado, 50,83, é cerca de 2,57 vezes o da ESPADA LONGA, 19,80, o que constitui um forte indício de desequilíbrio e justifica análise pela equipa de design.

6.3 Ficheiros gerados

Ficheiro	Conteúdo
jogadores_limpos.csv	Sete jogadores validados e normalizados.

Ficheiro	Conteúdo
ranking.csv	Média de DPS por arma.
log_rejeitados.txt	Registos rejeitados e respetivos motivos.

6.4 Dificuldades e soluções

Dificuldade	Solução aplicada
Lazy conversion causava erro ao tratar nulos.	A opção foi desativada no CSV file input.
Os lookups com o XML não encontravam correspondências.	Os identificadores provenientes do XML foram convertidos para Integer.
A tabela acumulava resultados de execuções anteriores.	Foi ativada a opção Truncate table.
Uma transformação podia apagar o log produzido pela outra.	Foi ativado Append nos dois passos de escrita.

7. Vídeo de demonstração

Figura 4 — QR Code do vídeo de demonstração

8. Conclusão e trabalhos futuros

O trabalho permitiu consolidar os principais conceitos de integração de dados através de um processo ETL completo.

A solução integrou três formatos de entrada, aplicou validação por expressão regular, tratamento de nulos, normalização, filtragem condicional, três lookups, cálculo e agregação. O resultado foi exportado para ficheiros, persistido em PostgreSQL e controlado por um Job com ramos explícitos de sucesso e falha.

A pergunta de negócio foi respondida: o MACHADO DE GUERRA apresenta o maior DPS médio e destaca-se claramente das restantes armas. Para além do resultado analítico, o processo garante rastreabilidade através do log de rejeições e evita duplicação da carga com Truncate table.

A utilização de caminhos relativos e de um repositório Git facilita ainda a portabilidade e a reprodução do projeto.

As principais limitações são o truncamento do DPS, a utilização de dados estáticos e a ausência de uma visualização dinâmica. Como trabalhos futuros propõe-se:

- Converter os operandos do DPS para decimal antes da divisão.
- Ligar Grafana, Node-RED ou Power BI ao PostgreSQL para apresentar um dashboard atualizado após cada execução.
- Receber telemetria em tempo real através de MQTT, Node-RED ou uma API REST.
- Agendar o Job e enviar ranking.csv por correio eletrónico ou SFTP.
- Separar explicitamente a solução em transformações de Extract, Transform e Load.
- Calcular novas métricas, como DPS por boss, região ou jogador.

9. Referências bibliográficas

Hitachi Vantara. (2026). Pentaho Data Integration 11.0. Disponível em: https://docs.pentaho.com/pdia-data-integration

PostgreSQL Global Development Group. (2026). PostgreSQL 18 Documentation. Disponível em: https://www.postgresql.org/docs/18/

Ferreira, L., & Ribeiro, Ó. (2026). Trabalho Prático I — Integração de Sistemas de Informação, Licenciatura em Engenharia de Sistemas Informáticos, ano letivo 2026/27. Enunciado da Unidade Curricular.